RAGenius Chatbot

Intelligent AI Agent with RAG, FAISS, and Multi-Memory Architecture

Author: Issam Adoch

GitHub: https://github.com/issam-mustapha
LinkedIn: www.linkedin.com/in/issam-ai-engineer

Technologies: LangChain, LangGraph, RAG, FAISS, FastAPI, OCR, Ollama, Mistral 7B, Next.js, TypeScript, Tailwind CSS, Axios, PostgreSQL

March 28, 2026

Introduction

RAGenius is an intelligent AI chatbot designed to provide a personalized, secure, and professional user experience. The system combines retrieval-augmented generation (RAG), FAISS for vector-based document search, and a multi-memory architecture to manage both short-term and long-term user context.

The chatbot allows users to upload documents and images, interact naturally, and receive accurate answers based on their personal context. The platform is designed with security, personalization, and usability in mind, offering a seamless interface for both authenticated and non-authenticated users.

User Experience

Connected Users

- Access to short-term and long-term memory.
- Upload PDF and image files for queries or summarization.
- Personalized experience based on preferences such as hobbies, language, profession, communication style, and learning methods.
- Modern and clear chatbot interface.
- View past conversations clearly.
- Secure logout functionality.

Non-Connected Users

- Interactions are limited to short-term memory only.
- Session ends clear all memory.
- Sign up via Google or form.
- Sign in via Google or form.
- Clear and concise answers based on temporary context.

Authentication and Security

- Highly secure token-based authentication.
- Middleware for PII detection to mask sensitive information before sending queries to LLM.
- Validation middleware ensures correctness of responses before delivering to the user.

Internal Architecture and Workflow

When a connected user sends a PDF:

- The file is processed into embeddings and stored in FAISS, isolated per user for security.
- Queries on the PDF use the specific uploaded file as the source.
- Queries not related to a single PDF retrieve information from all stored PDFs using similarity search.
- Summarization and analysis are performed using LLM tools with context-aware processing.
- Long-term memory is updated automatically with user preferences for improved personalization.

Use Case Diagram

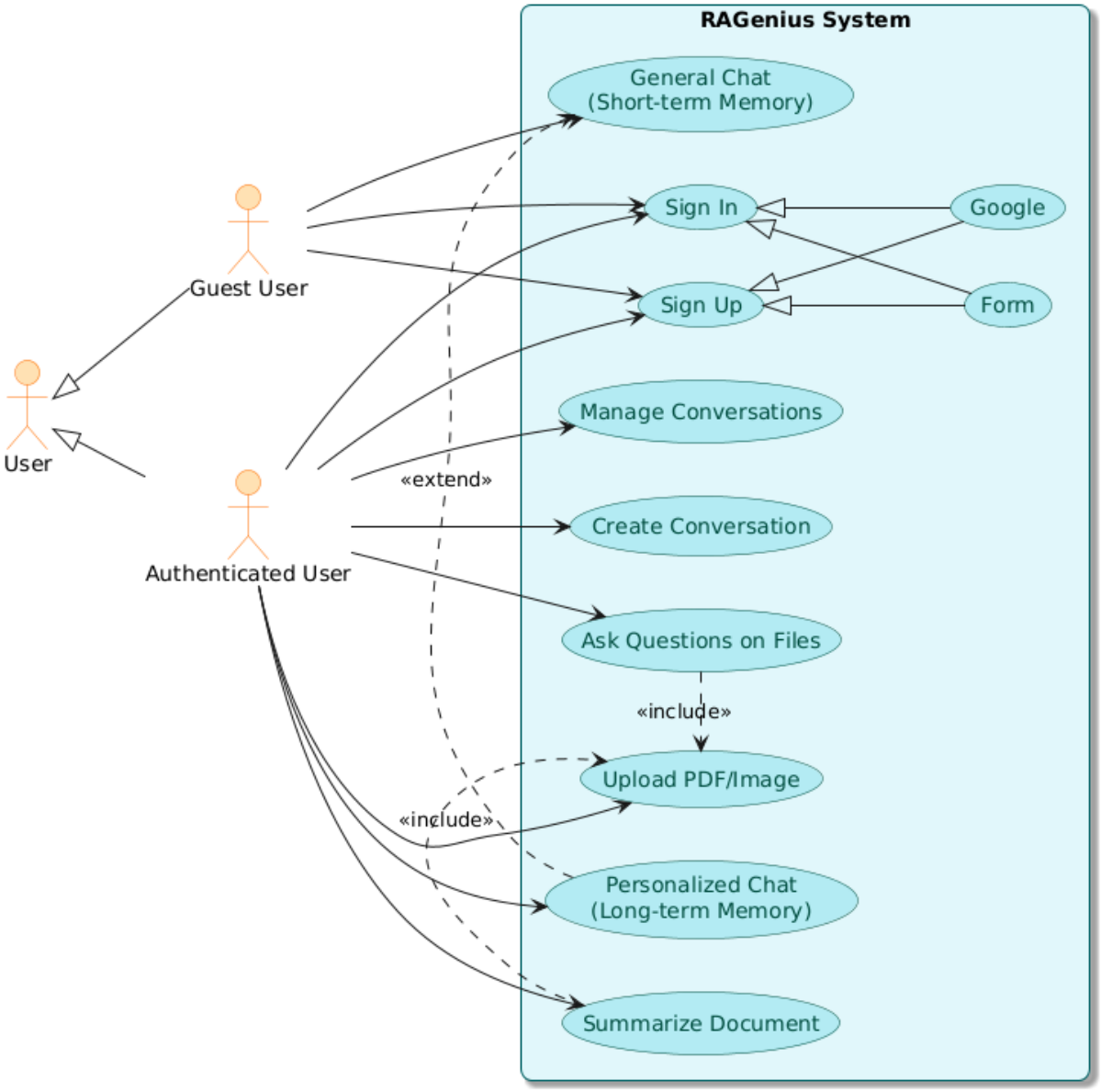

Figure 1: The diagram represents functionalities available for authenticated and non-authenticated users.

Architecture Flow Overview

- **PDF → Embeddings → FAISS**
- **Image → OCR → Cleaning Data**
- **Direct Question → LLM**
- RAG integration for context retrieval.
- Middleware ensures security and PII masking before sending queries to LLM.

- Short-Term Memory (RAM) stores conversation context.
- Long-Term Memory (PostgreSQL) stores historical interactions and preferences.
- Central Agent orchestrates RAG, memory, LLM, and response validation.

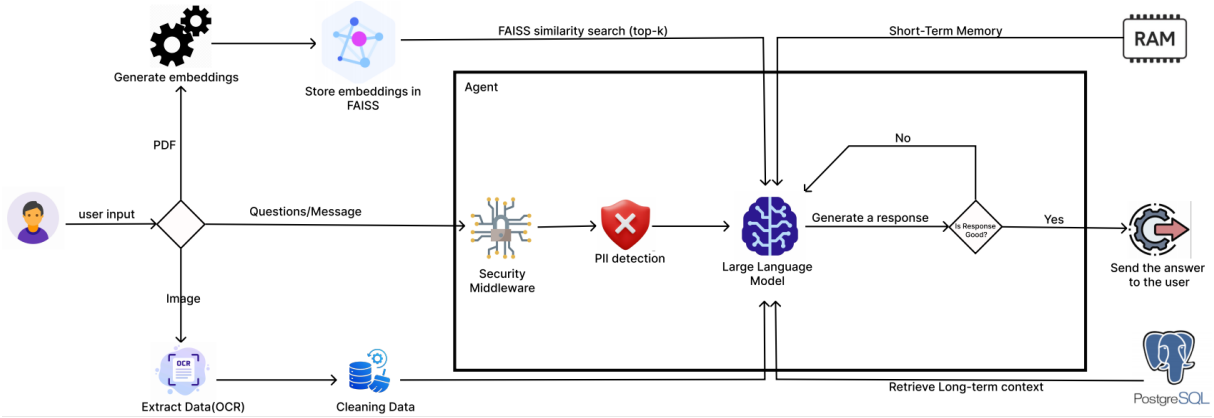

Figure 2: Overview of the AI agent architecture and workflow.

Backend Technologies

- **LangChain:** Platform for building intelligent and secure AI agents, combining reasoning, memory, and autonomous actions.
- **LangGraph:** Open-source framework to model stateful AI agents using nodes and edges for multi-step workflows and conditional decisions.
- **RAG:** Retrieval-Augmented Generation for combining document retrieval with language model generation.
- **FastAPI:** High-performance web framework for RESTful APIs.
- **OCR:** Converts text from images and scanned documents into machine-readable format.
- **Ollama:** Platform for managing local LLMs like Mistral 7B for offline AI execution.
- **Mistral 7B:** 7-billion parameter open-source LLM for document analysis and chatbot generation.

Frontend Technologies

- **Next.js:** Framework for modern React applications with SSR and SSG support.
- **TypeScript:** Superset of JavaScript enabling static typing for maintainability and error checking.
- **Tailwind CSS:** Utility-first CSS framework for fast, responsive UI design.
- **Axios:** Promise-based HTTP client for data fetching and API requests.

Development Tools

- **Git:** Distributed version control system for source code management.
- **GitHub:** Cloud platform for hosting Git repositories and collaboration.
- **Docker:** Containerization platform for consistent deployment.
- **VS Code:** Lightweight, cross-platform IDE.
- **Postman:** API testing and automation tool.
- **PostgreSQL:** Relational database for storing long-term memory and metadata.

Implementation

Chatbot Interface Pages

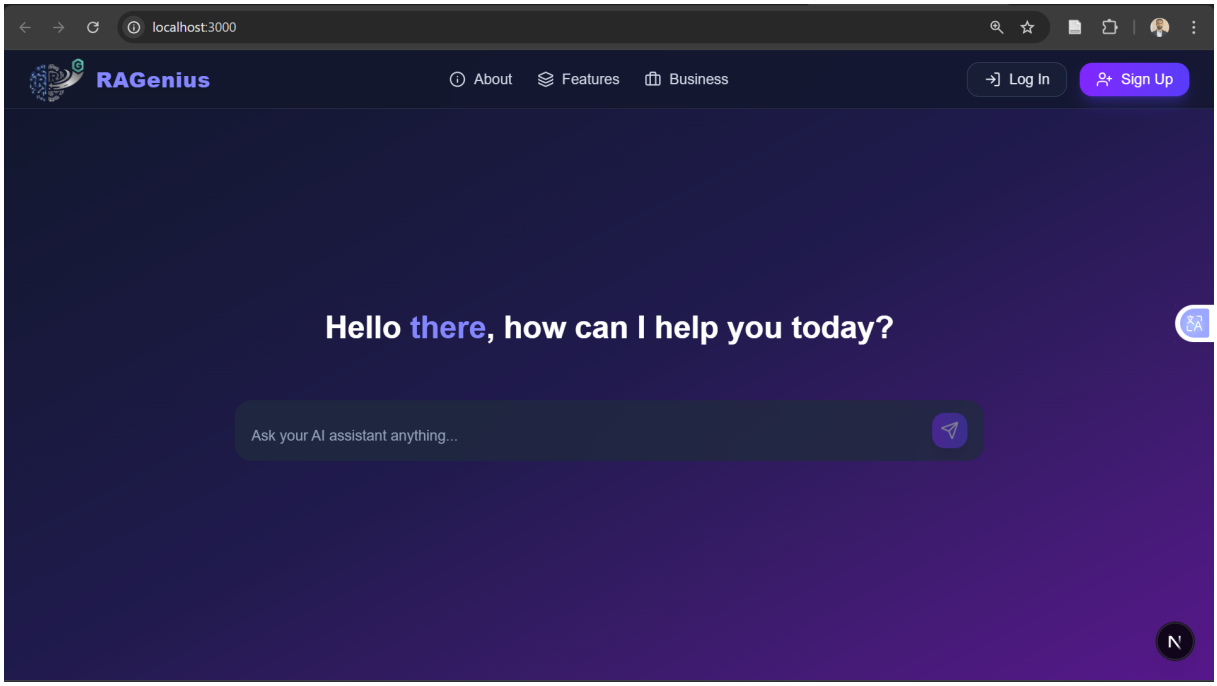

Figure 3: Non-authenticated users can interact with the chatbot using short-term memory. Responses are displayed temporarily, and users have the option to log in or sign up to access additional features.

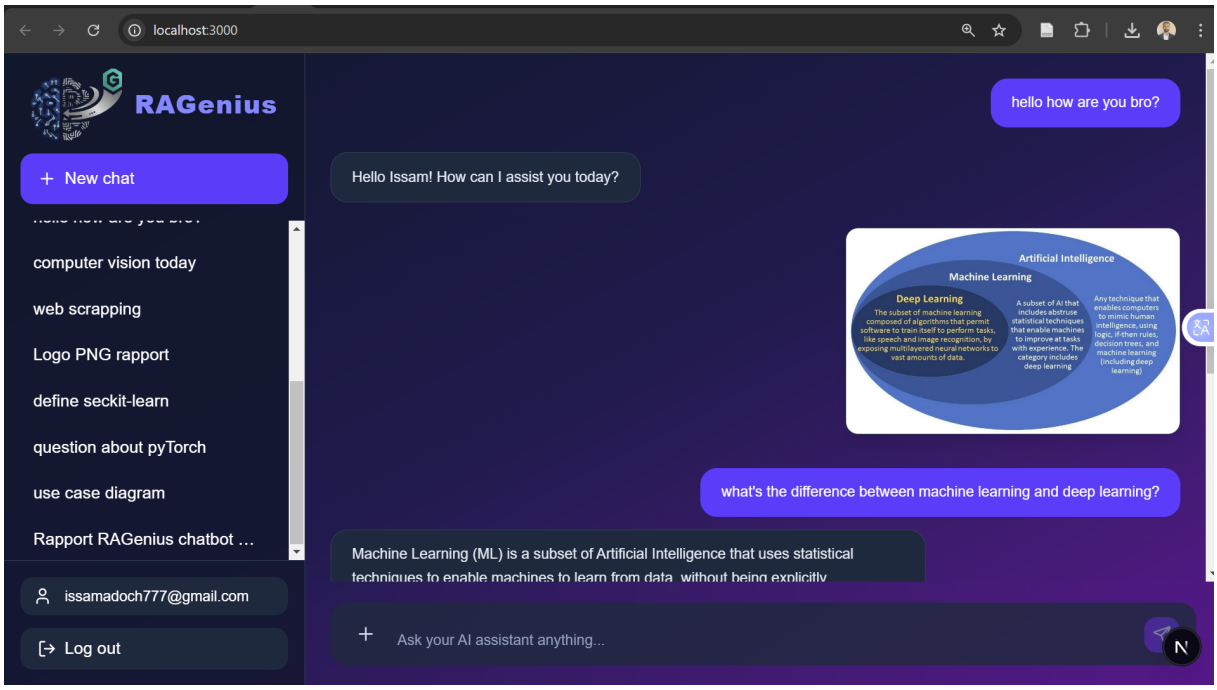

Figure 4: Authenticated users have full access to the chatbot functionalities, including interaction using short-term and long-term memory, uploading PDF or image files, creating new conversations, and viewing previous conversations in a personalized history.

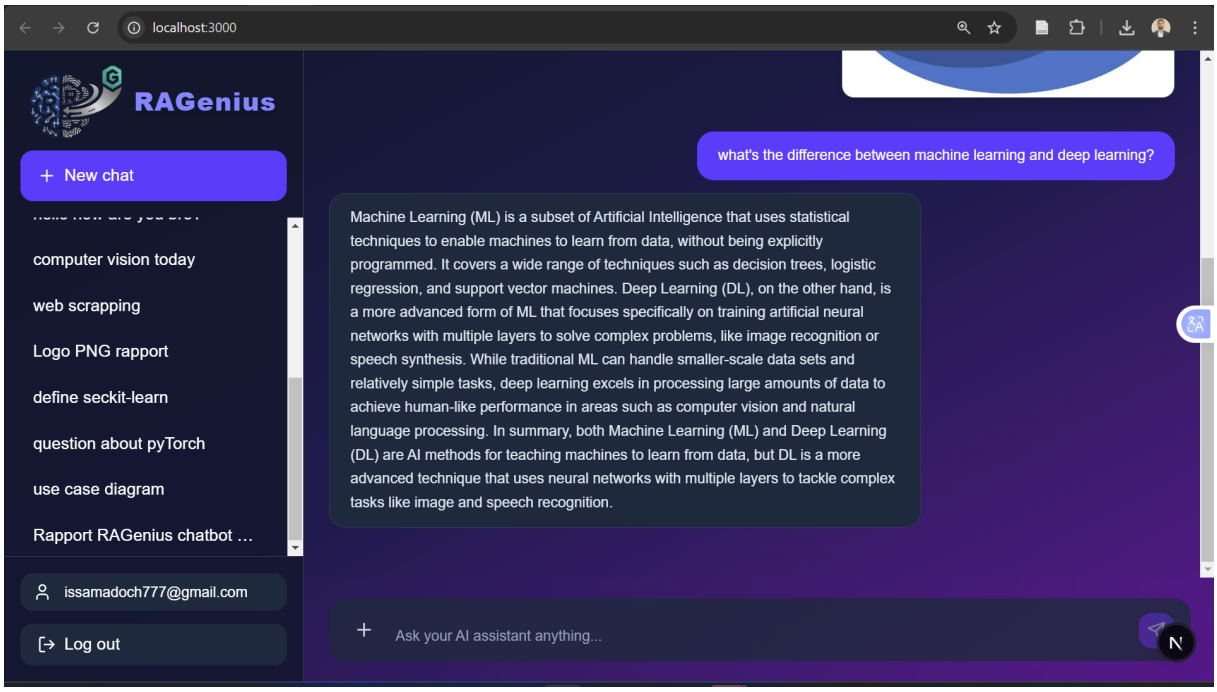

Figure 5: Continuation of the conversation interface for authenticated users. Users can continue previous chats, manage multiple conversations, and access conversation history seamlessly.

Authentication Pages

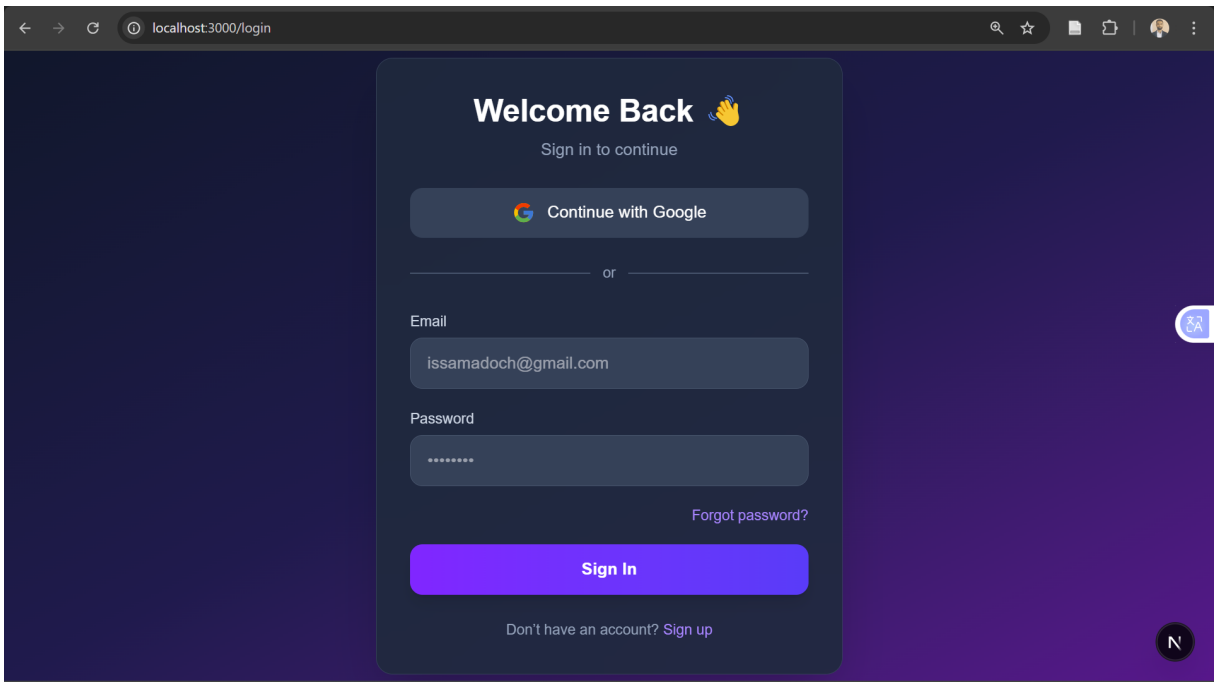

Figure 6: Users can securely log in using their email and password or via Google authentication. The system validates credentials, manages secure sessions, and grants access to personalized chatbot functionalities, including document upload and conversation history.

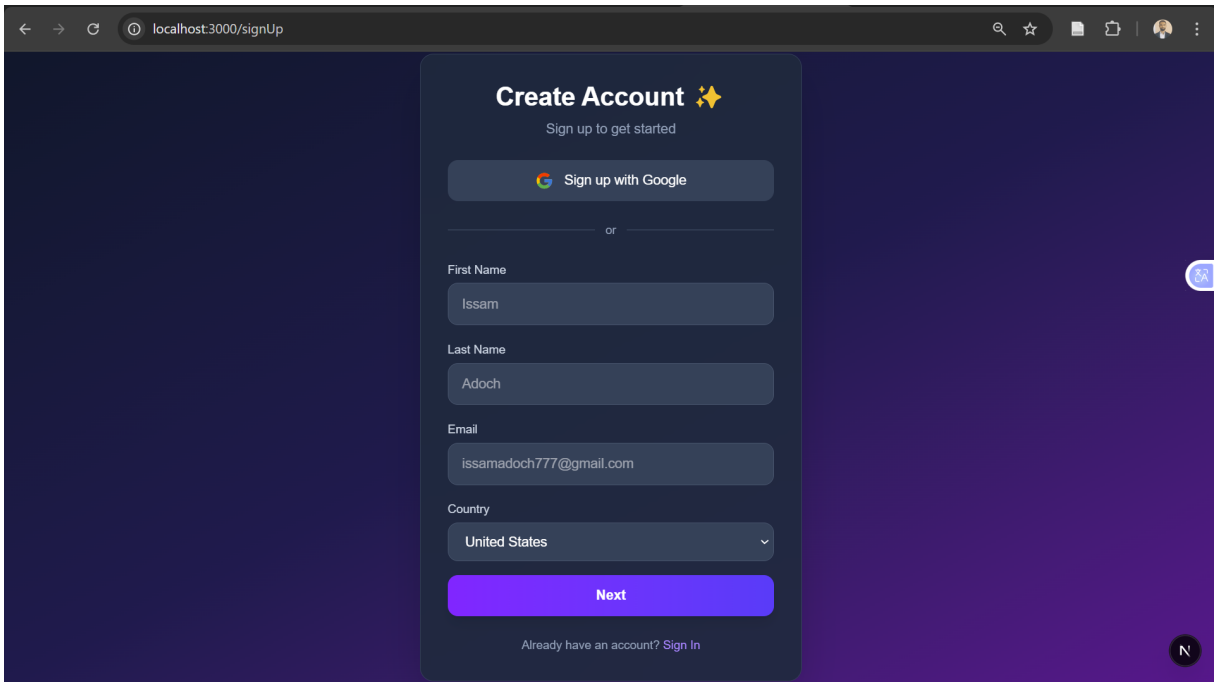

Figure 7: New users create an account by completing the registration form with first name, last name, email, and password, or by using Google authentication. After successful registration, users gain full access to all chatbot features, including document and image uploads, conversation creation, and personalized conversation history.

Conclusion

RAGenius Chatbot demonstrates a professional and modern AI agent architecture integrating RAG, FAISS, and multi-memory systems. It offers secure, personalized, and precise responses while maintaining a seamless user experience. The system is designed for scalability, robustness, and high usability, making it suitable for professional deployment with multiple user scenarios, from casual interactions to document-intensive queries.